

PROIECT LA DISCIPLINA PROGRAMARE ORIENTATĂ PE OBIECTE

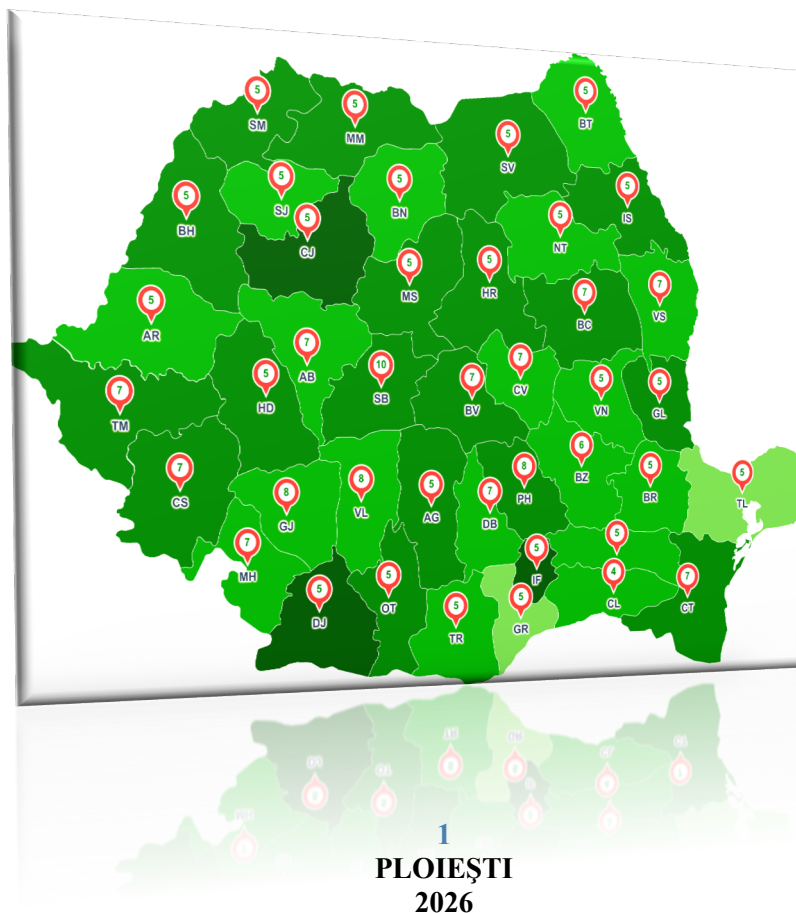
TEMĂ: Ghid Turistic România

Titular disciplină:

Șef lucr. dr. ing. Ioniță Irina

Studenti:

Dinu Bianca, Petrișor Raluca, Voicu Iosif





Cuprins

1. Formularea problemei	3
2. Etapele dezvoltării aplicației software - abordare UML	3
2.1. Analiza.....	3
2.2. Proiectarea	4
2.2.1. Identificarea actorilor	4
2.2.2. Identificarea cazurilor de utilizare.....	4
2.2.3. Diagramele de secvență sistem.....	8
2.2.4. Identificarea claselor	11
2.3. Implementarea	13
2.4. Testarea.....	14
3. Concluzii	17
4. Bibliografie	18
5. Anexe	19

1. Formularea problemei

Gestionarea și prezentarea obiectivelor turistice din România poate fi realizată eficient printr-o aplicație desktop orientată obiect, care îmbină o hartă interactivă, filtre tematice, căutare rapidă și persistență a preferințelor utilizatorului. Aplicația „Ghid Turistic România” permite utilizatorului să exploreze județele țării, să vizualizeze atracții turistice și să salveze obiectivele preferate.

Aplicația este construită în Java, cu interfață grafică JavaFX și bază de date SQLite. Din punct de vedere orientat obiect, sistemul separă modelul datelor (Atracție, Județ), accesul la date (DatabaseManager), manipularea hărții (MapHandler), controlul interfeței (MainController) și pornirea aplicației (HelloApplication).

Tabelul 1. Funcționalități principale ale aplicației

Sarcină minimală	Descriere în aplicație
Afișarea hărții României	Harta este compusă din imagini ale județelor, fiecare județ având număr de atracții afișat vizual.
Filtrare pe categorii	Utilizatorul poate selecta categoriile Istoric, Natură, Cultură, Religios și Agrement.
Căutare rapidă	Bara de căutare afișează rezultate în timp real din lista de atracții.
Sugestii aleatorii	Secțiunea „Descoperă România” schimbă periodic atracția propusă.
Detalii atracție	Panoul din dreapta afișează fotografie, tip, nume, rating, preț și descriere.
Favorite	Starea de favorit este salvată persistent în baza de date SQLite.
Personalizare fundal	Culoarea de fundal este salvată în Preferences și păstrată la redeschidere.

Scenariu general de funcționare:

1. Utilizatorul deschide aplicația și vizualizează harta României.
2. Utilizatorul filtrează atracțiile sau caută direct un obiectiv turistic după nume.
3. Sistemul afișează județul corespunzător, execută efectul de zoom și listează atracțiile disponibile.
4. Utilizatorul selectează o atracție și vizualizează detaliile în panoul lateral.
5. Utilizatorul poate adăuga atracția la Favorite și o poate regăsi ulterior în lista sa.

2. Etapele dezvoltării aplicației software - abordare UML

2.1. Analiza

În etapa de analiză au fost extrase cerințele relevante pentru domeniul aplicației: explorarea turistică, navigarea pe hartă, filtrarea datelor și salvarea preferințelor. Modelul abstract al aplicației se bazează pe obiecte care descriu atracții, județe, interacțiuni cu harta și operații asupra bazei de date.

Cerințe funcționale:

- Sistemul permite afișarea hărții interactive a României.
- Sistemul permite filtrarea atracțiilor turistice pe cinci categorii.
- Sistemul permite căutarea text și afișarea rezultatelor în timp real.

- Sistemul permite deschiderea unui județ și afișarea atracțiilor asociate.
- Sistemul permite vizualizarea detaliilor unei atracții turistice.
- Sistemul permite adăugarea/eliminarea unei atracții din lista de Favorite.
- Sistemul permite vizualizarea listei de Favorite.
- Sistemul permite schimbarea culorii de fundal și salvarea acestei preferințe.

Cerințe nefuncționale și constrângeri:

Tabelul 2. Cerințe nefuncționale

Categorie	Cerință / constrângere
Interfață grafică	Aplicația folosește JavaFX și fișier FXML pentru definirea layout-ului.
Persistență	Datele despre atracții, județe și starea Favorite sunt stocate în baza de date SQLite ghid_turistic.db.
Portabilitate	Aplicația poate rula pe sisteme care au instalat Java 17+ și dependențele JavaFX.
Separarea responsabilităților	Clasele au roluri distincte: model, acces la date, controler, manipulare hartă și lansare aplicație.
Utilizare	Interfața este orientată către un turist fără cunoștințe tehnice, cu butoane și panouri vizuale.

2.2. Proiectarea

2.2.1. Identificarea actorilor

Tabelul 3. Actorii sistemului

Actor	Tip actor	Rol în aplicație
Turist / Utilizator	Actor extern	Persoana care folosește aplicația pentru a căuta atracții turistice, a filtra obiective, a vizualiza detalii și a salva favorite.

2.2.2. Identificarea cazurilor de utilizare

Tabelul 4. Cazurile de utilizare

No.	Caz de utilizare	Actor	Descriere
1	Căutare atracție turistică	Turist	Utilizatorul caută un obiectiv după nume, iar sistemul îl localizează pe hartă.
2	Filtrare obiective pe categorii	Turist	Utilizatorul selectează una sau mai multe categorii, iar harta și listele sunt actualizate.
3	Vizualizare detalii și hartă județ	Turist	Utilizatorul deschide un județ sau o atracție și vede detaliile asociate.
4	Adăugare / eliminare din Favorite	Turist	Utilizatorul schimbă starea de favorit a unei atracții.
5	Vizualizare listă Favorite	Turist	Utilizatorul consultă atracțiile marcate anterior ca favorite.

6	Schimbare fundal aplicație	Turist	Utilizatorul personalizează culoarea fundalului hărții.
---	----------------------------	--------	---

Diagrama cazurilor de utilizare

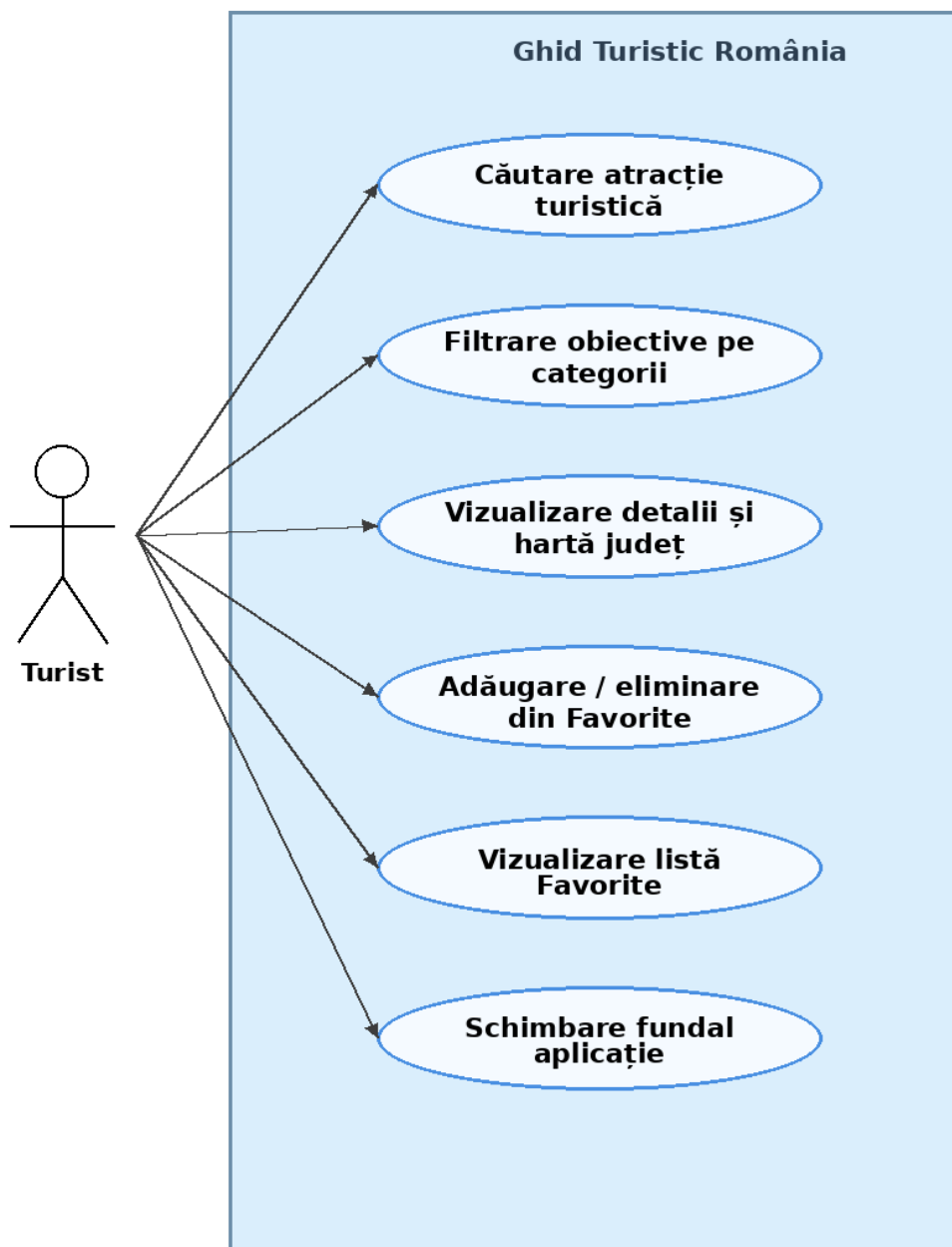


Figura 1. Diagrama cazurilor de utilizare

Tabelul 5. Fișa tip a cazului de utilizare Căutare atracție turistică

Fișa-tip	Căutare atracție turistică
Actori	Turist
Obiectiv	Localizarea rapidă a unei atracții turistice după nume.
Precondiții	Aplicația este deschisă, iar lista de atracții a fost încărcată din baza de date.
Scenariul nominal	1. Utilizatorul tastează în bara de căutare. 2. Sistemul filtrează în timp real lista de atracții. 3. Utilizatorul selectează rezultatul dorit. 4. Sistemul execută zoom pe județ și deschide panoul de detalii.
Extensii	Dacă nu există rezultate, lista rămâne goală și utilizatorul poate șterge textul căutării.
Postcondiții	Atracția selectată este afișată în interfață.

Tabelul 6. Fișa tip a cazului de utilizare Filtrare obiective pe categorii

Fișa-tip	Filtrare obiective pe categorii
Actori	Turist
Obiectiv	Afișarea obiectivelor turistice doar din categoriile selectate.
Precondiții	Harta este afișată și butoanele de categorie sunt disponibile.
Scenariul nominal	1. Utilizatorul selectează una sau mai multe categorii. 2. Sistemul preia filtrele active. 3. Sistemul recalculează numărul de atracții pe județe. 4. Dacă un județ sau lista Favorite este deschisă, conținutul acestuia se actualizează.
Extensii	Dacă nu este selectată nicio categorie, sistemul afișează toate atracțiile.
Postcondiții	Harta și listele respectă filtrele selectate.

Tabelul 7. Fișa tip a cazului de utilizare Vizualizare detalii și hartă județ

Fișa-tip	Vizualizare detalii și hartă județ
Actori	Turist
Obiectiv	Prezentarea atracțiilor dintr-un județ și a detaliilor unui obiectiv.
Precondiții	Există județe și atracții în baza de date.
Scenariul nominal	1. Utilizatorul apasă pe un județ de pe hartă. 2. MapHandler execută efectul de zoom. 3. MainController cere atracțiile județului din DatabaseManager. 4. Sistemul afișează cartonașele atracțiilor în panoul stâng. 5. Utilizatorul selectează o atracție. 6. Sistemul afișează imaginea, descrierea, ratingul și prețul în panoul din dreapta listei atracțiilor.
Extensii	Dacă județul nu are atracții pentru filtrele active, sistemul afișează mesaj informativ.
Postcondiții	Atracția selectată este evidențiată pe hartă și în listă.

Tabelul 8. Fișa tip a cazului de utilizare Adăugare / eliminare din Favorite

Fișa-tip	Adăugare / eliminare din Favorite
Actori	Turist
Obiectiv	Salvarea sau eliminarea unei atracții din lista de preferințe a utilizatorului.
Precondiții	Panoul de detalii al unei atracții este deschis.
Scenariul nominal	1. Utilizatorul apasă butonul cu pictograma inimă. 2. Obiectul Atracție își inversează starea favorit. 3. MainController apelează updateFavorit în DatabaseManager. 4. Baza de date actualizează valoarea câmpului Favorit. 5. Pictograma inimii se schimbă în interfață.
Extensii	Dacă apare o eroare SQL, mesajul este afișat în consolă.
Postcondiții	Starea de favorit este salvată persistent.

Tabelul 9. Fișa tip a cazului de utilizare Vizualizare listă Favorite

Fișa-tip	Vizualizare listă Favorite
Actori	Turist
Obiectiv	Afișarea tuturor atracțiilor marcate ca favorite.
Precondiții	Există sau pot exista atracții marcate ca favorite în baza de date.
Scenariul nominal	1. Utilizatorul apasă „Vezi favoritele mele”. 2. Sistemul citește toate atracțiile din baza de date. 3. Sistemul păstrează doar atracțiile cu favorit=true. 4. Sistemul aplică filtrele active, dacă există. 5. Panoul stâng afișează cartonașele favorite.
Extensii	Dacă nu există favorite pentru filtrele curente, sistemul afișează un mesaj informativ.
Postcondiții	Lista de favorite este vizibilă în interfață.

Tabelul 10. Fișa tip a cazului de utilizare Schimbare fundal aplicație

Fișa-tip	Schimbare fundal aplicație
Actori	Turist
Obiectiv	Personalizarea vizuală a fundalului hărții.
Precondiții	Butonul de culoare este disponibil în panoul de start.
Scenariul nominal	1. Utilizatorul apasă una dintre culorile disponibile. 2. MainController setează stilul CSS al containerului hărții. 3. Culoarea este salvată în Preferences. 4. La redeschiderea aplicației, culoarea salvată este reîncărcată.
Extensii	Dacă nu există preferință salvată, se folosește culoarea implicită #e8f4f8.
Postcondiții	Fundalul aplicației are culoarea aleasă de utilizator.

2.2.3. Diagramele de secvență sistem

Diagramele de secvență de mai jos descriu interacțiunea dintre actorul extern și principalele componente ale aplicației pentru fiecare caz de utilizare considerat.

DSS - Căutare atracție turistică

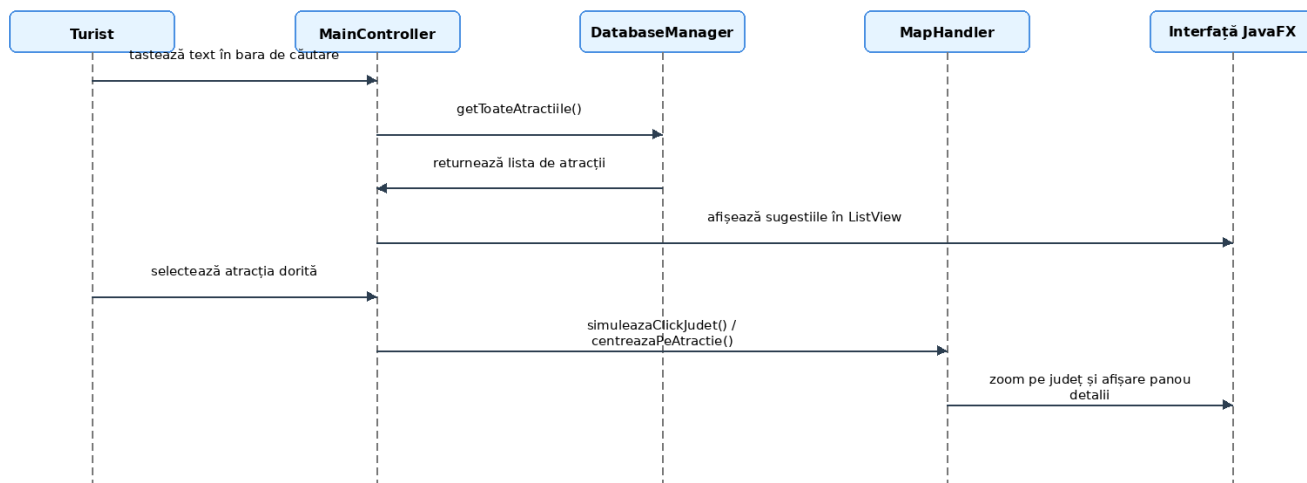


Figura 2. DSS Căutare atracție turistică

DSS - Filtrare obiective pe categorii

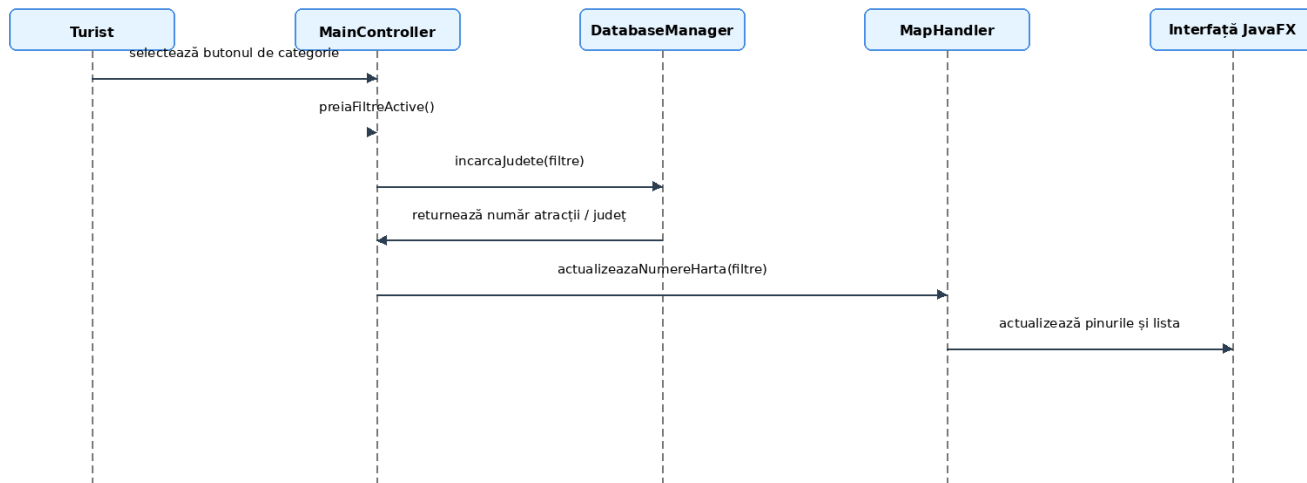


Figura 3. DSS Filtrare obiective pe categorii

DSS - Vizualizare detalii și hartă județ

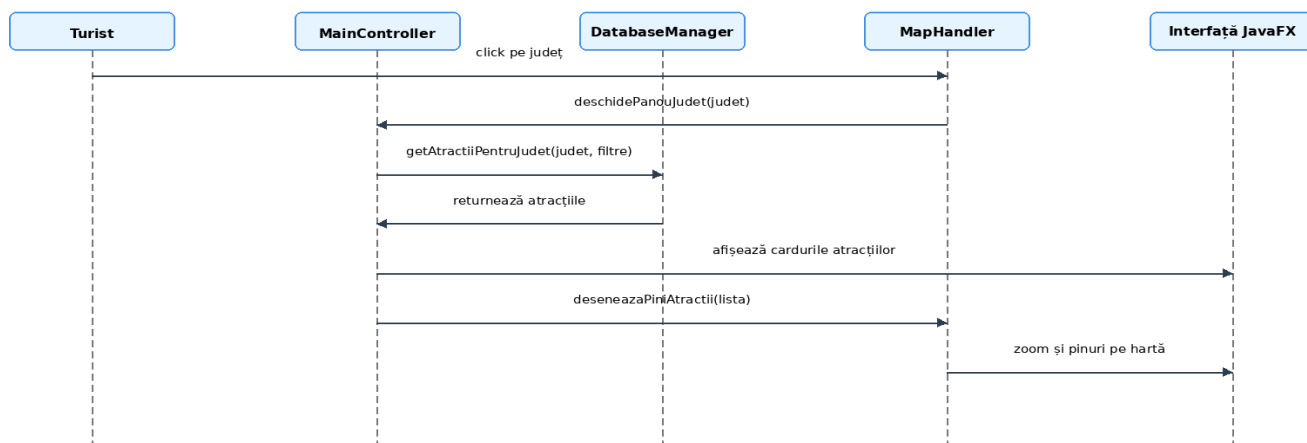


Figura 4. DSS Vizualizare detalii și hartă județ

DSS - Adăugare / eliminare din Favorite

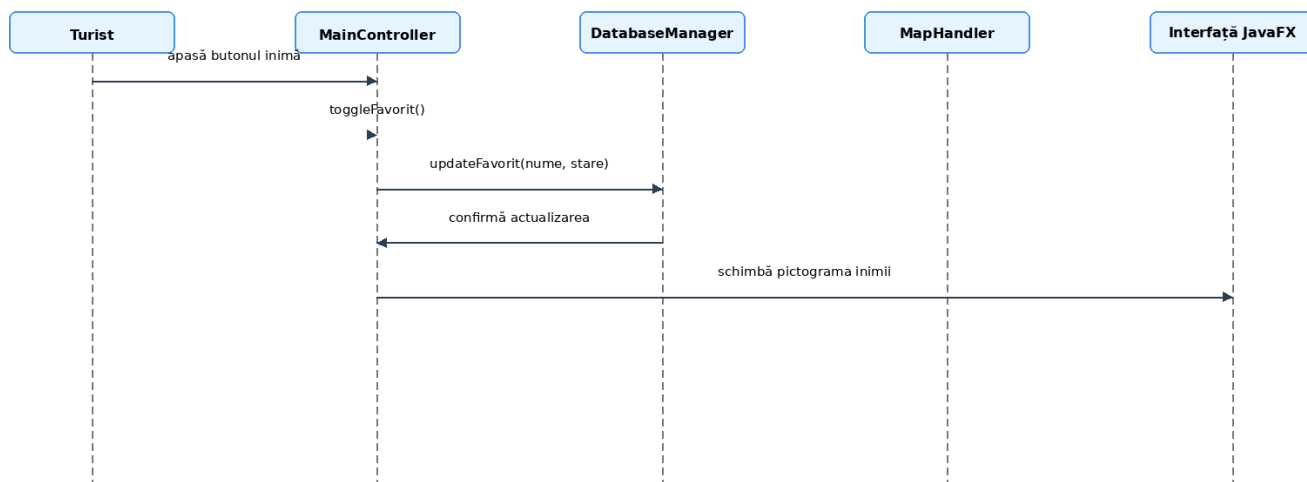


Figura 5. DSS Adăugare / eliminare din Favorite

DSS - Vizualizare listă Favorite

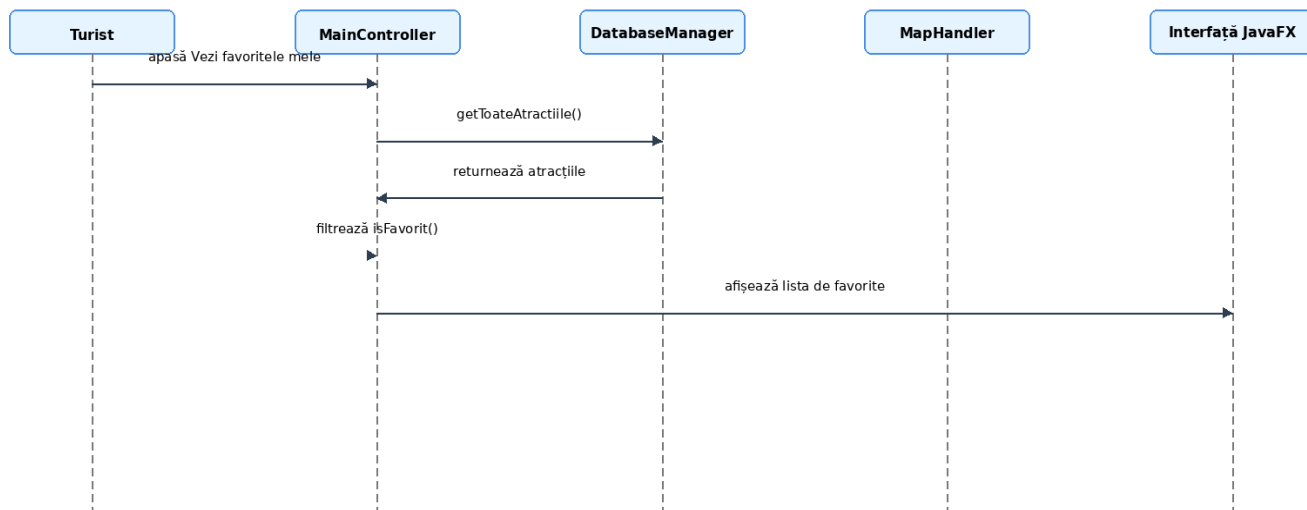


Figura 6. DSS Vizualizare listă Favorite

DSS - Schimbare fundal aplicație

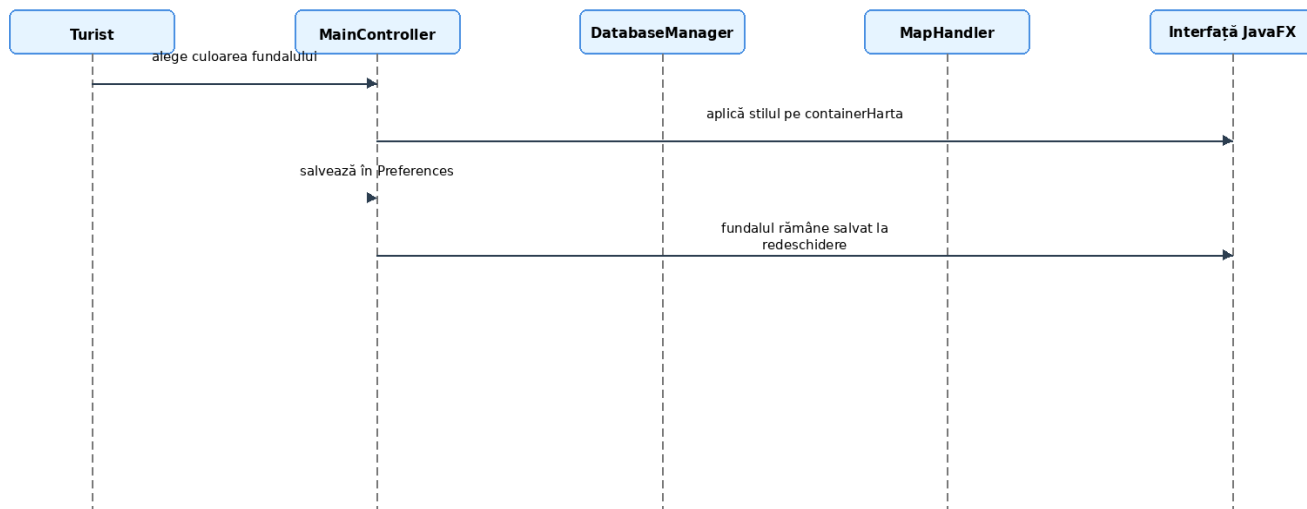


Figura 7. DSS Schimbare fundal aplicație

2.2.4. Identificarea claselor

Tabelul 11. Clasele și componentele identificate

Clasă / fișier	Tip	Responsabilitate
Atractie	Model de date	Reține informațiile despre un obiectiv turistic: nume, județ, tip, poză, descriere, rating, preț, coordonate relative și starea de favorit.
Judet	Model de date	Reține informațiile grafice și logice despre județ: nume, cale imagine, poziție, dimensiune și număr de atracții.
DatabaseManager	Acces la date	Gestionează conexiunea SQLite, încărcarea județelor, încărcarea atracțiilor și actualizarea stării Favorite.
MapHandler	Logică hartă	Construiește harta, aplică zoom, desenează pini, tooltip-uri și centrează atracția selectată.
MainController	Controler JavaFX	Leagă interfața de baza de date și de hartă: search, filtre, panouri, favorite, fundal.
HelloApplication	Clasă principală	Pornește aplicația JavaFX, încarcă fișierul FXML și afișează fereastra.

Diagrama de clase

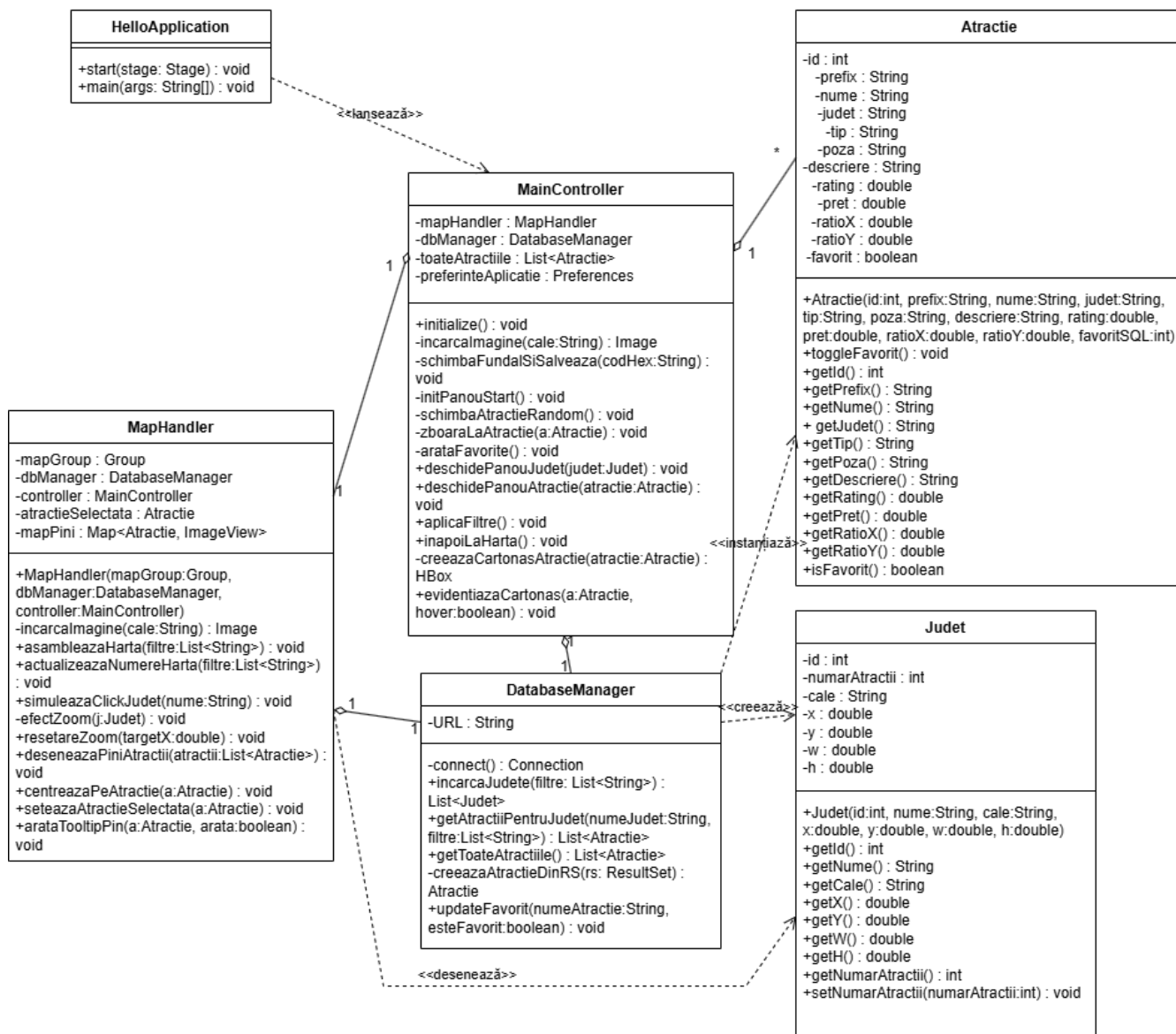


Figura 8. Diagrama de clase

2.3. Implementarea

Aplicația a fost dezvoltată în Java, folosind framework-ul JavaFX pentru interfața grafică și SQLite pentru stocarea datelor. Arhitectura urmărește principiile programării orientate pe obiecte: încapsulare prin attribute private, reutilizare prin clase de model, separarea responsabilităților între interfață, hărți și acces la date.

Tabelul 12. Tehnologii și componente de implementare

Componentă	Tehnologie / element folosit	Rol
Interfață grafică	JavaFX + hello-view.fxml	Construirea ferestrei principale și a panourilor interactive.
Persistență	SQLite + JDBC	Stocarea județelor, atracțiilor și a câmpului Favorit.
Model date	Atracție, Județ	Obiecte Java care descriu entitățile aplicației.
Control interfață	MainController	Evenimente, căutare, filtre, favorite și panouri.
Hartă	MapHandler	Afișare județe, zoom, pini și tooltip-uri.
Preferințe	java.util.prefs.Preferences	Salvarea culorii de fundal între sesiuni.

2.3.1. Selecție de cod sursă

Atracție - încapsularea datelor și schimbarea stării Favorite

```
public class Atracție {
    private int id;
    private String prefix, nume, judet, tip, poza, descriere;
    private double rating, pret, ratioX, ratioY;
    private boolean favorit;

    public void toggleFavorit() {
        this.favorit = !this.favorit;
    }

    public boolean isFavorit() {
        return favorit;
    }
}
```

DatabaseManager - actualizarea stării Favorite în SQLite

```
public void updateFavorit(String numeAtracție, boolean esteFavorit) {
    try (Connection conn = connect();
        PreparedStatement pstmt = conn.prepareStatement(
            "UPDATE atractii SET Favorit = ? WHERE Nume = ?")) {
        pstmt.setInt(1, esteFavorit ? 1 : 0);
        pstmt.setString(2, numeAtracție);
        pstmt.executeUpdate();
    } catch (SQLException e) {
        System.out.println("Eroare DB: " + e.getMessage());
    }
}
```

MainController - aplicarea filtrelor active

```
@FXML public void aplicaFiltre() {
    Arrays.asList(btnIstoric, btnNatura, btnCultura, btnReligios, btnAgrement)
        .forEach(this::actualizeazaStilButon);
}
```

```
List<String> filtre = preiaFiltreActive();
if (mapHandler != null) {
    mapHandler.actualizeazaNumereHarta(filtre);
}
}
```

MapHandler - asamblarea hărții din județe

```
public void asambleazaHarta(List<String> filtre) {
    mapGroup.getChildren().clear();
    for (Judet judet : dbManager.incarcaJudete(filtre)) {
        ImageView img = new ImageView(incarcaImagine("imagini/" + judet.getCale()));
        img.setLayoutX(judet.getX());
        img.setLayoutY(judet.getY());
        img.setFitWidth(judet.getWidth());
        img.setFitHeight(judet.getHeight());
        img.setOnMouseClicked(e -> {
            efectZoom(judet);
            controller.deschidePanouJudet(judet);
        });
        mapGroup.getChildren().add(img);
    }
}
```

2.4. Testarea

Testarea aplicației a fost organizată prin scenarii de utilizare. Pentru fiecare scenariu s-au urmărit pașii efectuați de utilizator, răspunsul sistemului și rezultatul așteptat.

Studiul de caz 1: Filtrare și Favorite

1. Utilizatorul apasă butonul „Natură”.
2. Sistemul actualizează numerele de pe hartă conform filtrului selectat.

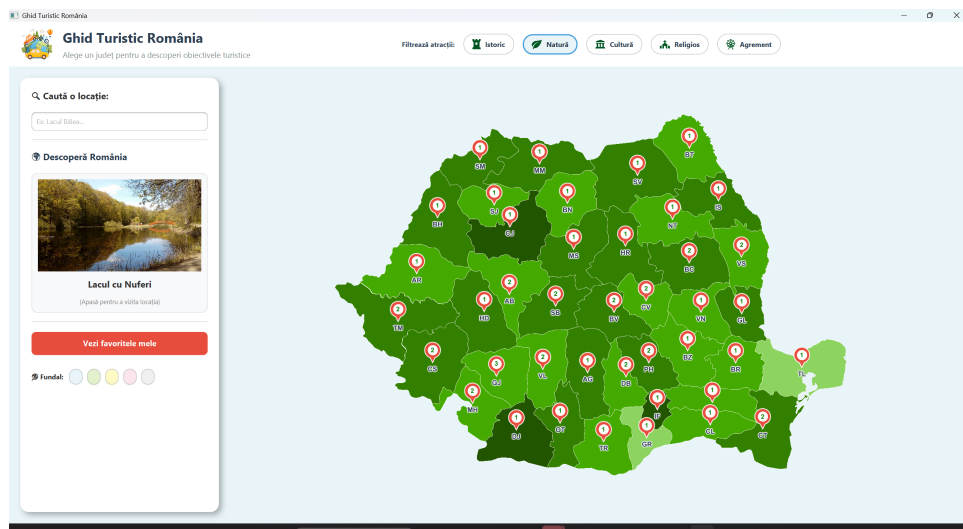


Figura 9: filtrul Natură activ și harta actualizată

3. Utilizatorul selectează un județ și apoi o atracție.
4. Utilizatorul apasă butonul inimă pentru salvarea în Favorite.

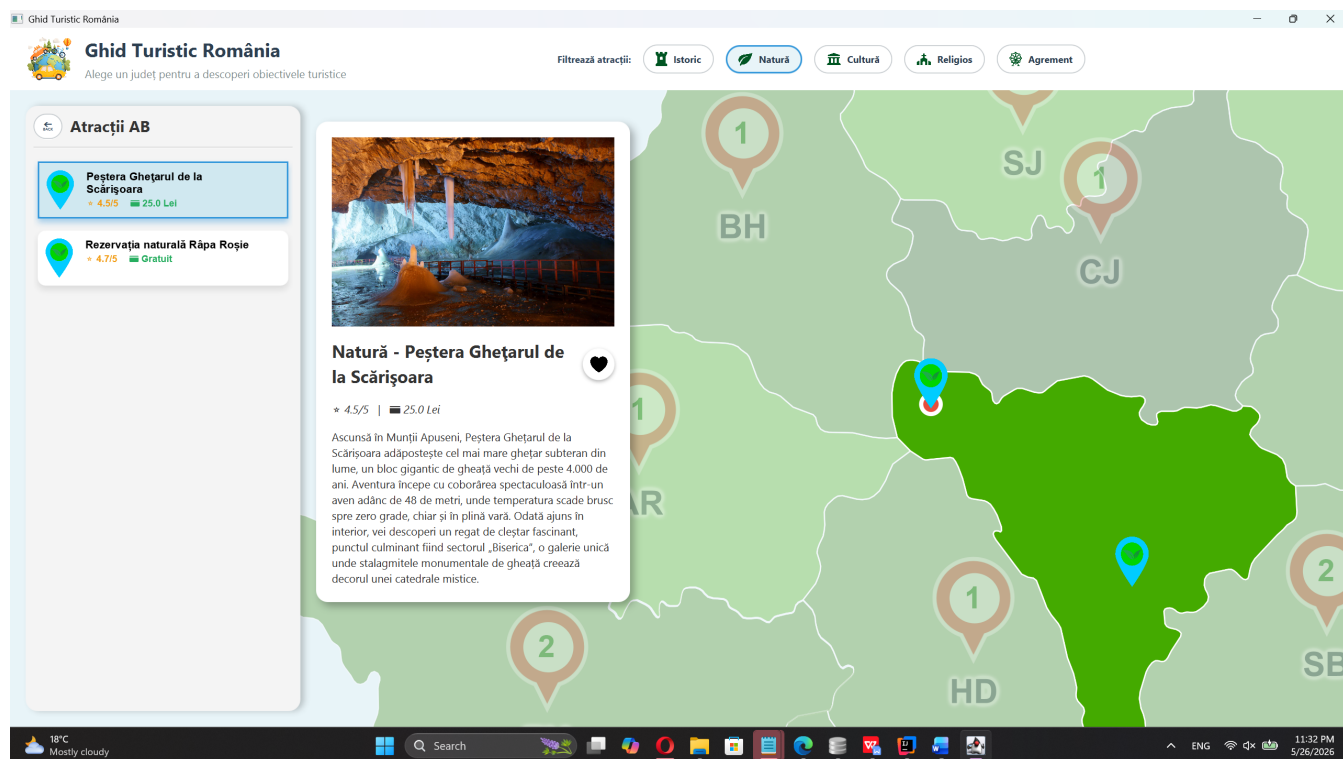


Figura 10: atracție deschisă și buton Favorite apăsate

- Utilizatorul apasă „Vezi favoritele mele” și verifică apariția atracției.

Studiul de caz 2: Căutarea unei locații

1. Utilizatorul tastează numele unei atracții, de exemplu „Castelul Corvinilor”.
2. Lista de căutare afișează rezultatele care conțin textul introdus.

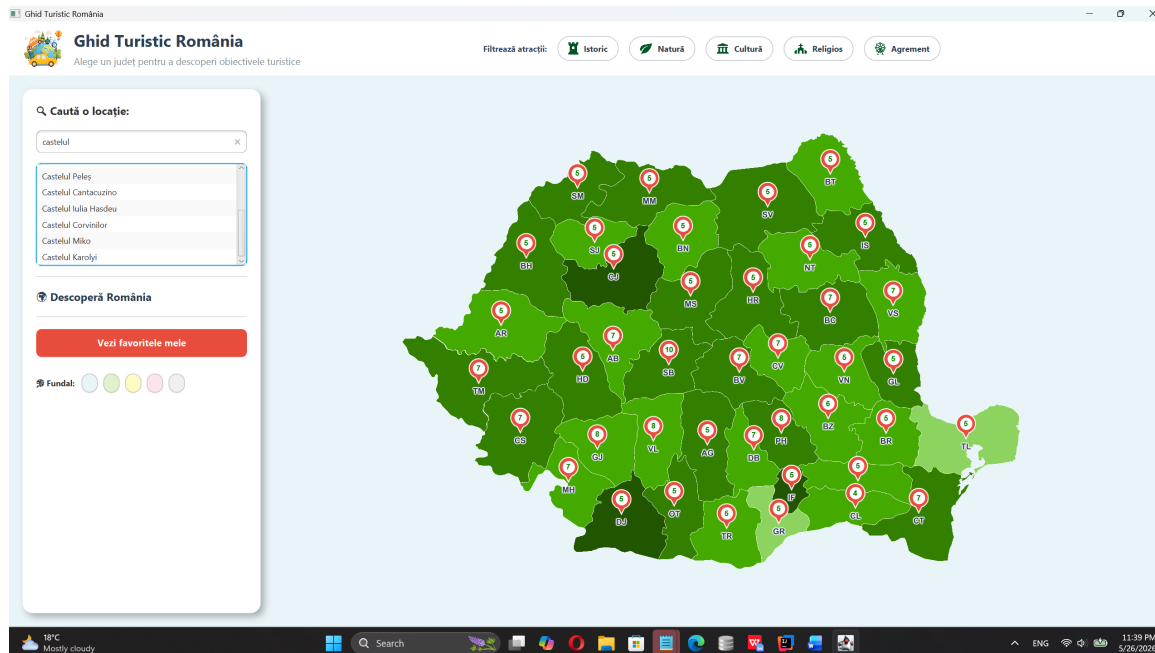


Figura 11: rezultat afișat în lista de căutare

3. Utilizatorul selectează rezultatul dorit.
4. Sistemul execută zoom pe județ și afișează detaliile atracției.

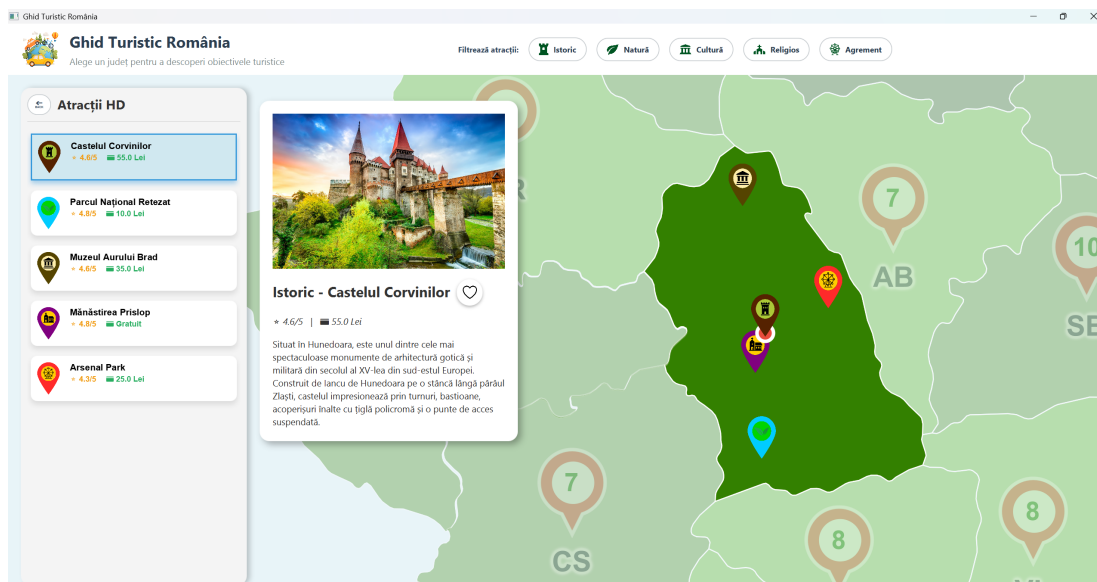


Figura 12: panou detalii deschis după selectarea rezultatului

3. Concluzii

Proiectul „Ghid Turistic România” demonstrează aplicarea programării orientate pe obiecte într-o aplicație desktop cu interfață grafică. Prin împărțirea aplicației în clase cu responsabilități clare, codul devine mai ușor de urmărit, extins și testat.

Avantaje ale aplicației:

- Separarea logicii de acces la date de logica interfeței prin clasa DatabaseManager.
- Utilizarea clasei MapHandler pentru operații specifice hărții, fără încărcarea excesivă a controlerului.
- Persistența datelor prin SQLite, fără necesitatea unui server extern.
- Interfață intuitivă prin JavaFX: panouri, căutare, imagini, filtre și butoane vizuale.
- Salvarea preferințelor de fundal prin Preferences, ceea ce personalizează experiența utilizatorului.

Dezavantaje / limitări:

- Nu există încă un modul de administrator pentru adăugarea atracțiilor direct din aplicație.
- Erorile SQL sunt afișate în consolă, nu într-un dialog grafic pentru utilizator.
- Aplicația este desktop și nu are sincronizare online între utilizatori.
- Actualizarea bazei de date se face local.

Direcții de îmbunătățire:

- Adăugarea unui modul de administrare cu autentificare.
- Adăugarea recenziilor și comentariilor utilizatorilor.
- Integrarea unei hărți reale online sau a rutelor de navigație.
- Exportarea listei de Favorite într-un fișier PDF sau text.
- Îmbunătățirea tratării erorilor prin mesaje grafice JavaFX.

4. Bibliografie

Eck, D. J. (2019). *Introduction to programming using Java* (Version 8.1).

<http://math.hws.edu/javanotes/>

Ioniță, L., et al. (2015). *Diagramele UML 2: Dicționar. Studii de caz. Aplicație web*. Editura Universității Petrol-Gaze din Ploiești.

OpenJFX. (n.d.). *OpenJFX documentation*. Retrieved June 1, 2026, from <https://openjfx.io/>

Oracle. (n.d.). *Object-oriented programming concepts*. The Java Tutorials. Retrieved June 1, 2026, from <https://docs.oracle.com/javase/tutorial/java/concepts/index.html>

Oracle. (n.d.). *Using FXML to create a user interface*. JavaFX Documentation. Retrieved June 1, 2026, from https://docs.oracle.com/javafx/2/get_started/fxml_tutorial.htm

Preda, M., Mircea, A., Preda, D., & Teodorescu, C. (2010). *Introducere în programarea orientată-obiect: Concepte fundamentale din perspectiva ingineriei software*. Editura Polirom.

SQLite. (n.d.). *SQLite documentation*. Retrieved June 1, 2026, from <https://www.sqlite.org/docs.html>

Visual Paradigm. (n.d.). *UML class diagram tutorial*. Retrieved June 1, 2026, from <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-class-diagram-tutorial/>

Xerial. (n.d.). *SQLite JDBC driver*. GitHub. Retrieved June 1, 2026, from <https://github.com/xerial/sqlite-jdbc>

Anexe:

Cod sursă - selecție pe clase

Codul sursă integral se predă electronic împreună cu proiectul. Mai jos sunt prezentate fragmentele relevante pentru clasele identificate în proiectare.

Atractie.java

```
package com.example.ghidturistic;

public class Atractie {
    private int id;
    private String prefix, nume, judet, tip, poza, descriere;
    private double rating, pret, ratioX, ratioY;
    private boolean favorit;

    public Atractie(int id, String prefix, String nume, String judet, String tip, String poza,
        String descriere, double rating, double pret, double ratioX, double ratioY, int favoritSQL) {
        this.id = id; this.prefix = prefix; this.nume = nume; this.judet = judet;
        this.tip = tip; this.poza = poza; this.descriere = descriere;
        this.rating = rating; this.pret = pret; this.ratioX = ratioX; this.ratioY = ratioY;
        this.favorit = (favoritSQL == 1);
    }

    public void toggleFavorit() { this.favorit = !this.favorit; }

    public int getId() { return id; }
    public String getPrefix() { return prefix != null ? prefix : ""; }
    public String getNume() { return nume; }
    public String getJudet() { return judet; }
    public String getTip() { return tip; }
    public String getPoza() { return poza; }
    public String getDescriere() { return descriere; }
    public double getRating() { return rating; }
    public double getPret() { return pret; }
    public double getRatioX() { return ratioX; }
    public double getRatioY() { return ratioY; }
    public boolean isFavorit() { return favorit; }
}
```

DatabaseManager.java

```
package com.example.ghidturistic;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class DatabaseManager {
    private static final String URL = "jdbc:sqlite:ghid_turistic.db";

    private Connection connect() throws SQLException {
        return DriverManager.getConnection(URL);
    }
}
```

```
public List<Judet> incarcaJudete(List<String> filtre) {
    List<Judet> lista = new ArrayList<>();
    String conditie = (filtre != null && !filtre.isEmpty()) ? " AND atractii.Tip IN (" + String.join(",", filtre) + ")" : "";
    String sql = "SELECT *, (SELECT COUNT(*) FROM atractii WHERE atractii.Judet = judete.ume" + conditie + ") AS total FROM judete";

    try (Connection conn = connect(); PreparedStatement pstmt = conn.prepareStatement(sql); ResultSet rs = pstmt.executeQuery()) {
        while (rs.next()) {
            Judet j = new Judet(rs.getInt("id"), rs.getString("nume"), rs.getString("cale"),
                rs.getDouble("x"), rs.getDouble("y"), rs.getDouble("w"), rs.getDouble("h"));
            j.setNumarAtractii(rs.getInt("total"));
            lista.add(j);
        }
    } catch (SQLException e) { System.out.println("Eroare DB: " + e.getMessage()); }
    return lista;
}

public List<Atractie> getAtractiiPentruJudet(String numeJudet, List<String> filtre) {
    List<Atractie> atractii = new ArrayList<>();
    String sql = "SELECT * FROM atractii WHERE Judet = ?" +
        ((filtre != null && !filtre.isEmpty()) ? " AND Tip IN (" + String.join(",", filtre) + ")" : "");

    try (Connection conn = connect(); PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setString(1, numeJudet);
        ResultSet rs = pstmt.executeQuery();
        while (rs.next()) atractii.add(creeazaAtractieDinRS(rs));
    } catch (SQLException e) { System.out.println("Eroare DB: " + e.getMessage()); }
    return atractii;
}

public List<Atractie> getToateAtractiile() {
    List<Atractie> atractii = new ArrayList<>();
    try (Connection conn = connect(); PreparedStatement pstmt = conn.prepareStatement("SELECT * FROM atractii"); ResultSet rs =
pstmt.executeQuery()
        while (rs.next()) atractii.add(creeazaAtractieDinRS(rs));
    } catch (SQLException e) { System.out.println("Eroare DB: " + e.getMessage()); }
    return atractii;
}

private Atractie creeazaAtractieDinRS(ResultSet rs) throws SQLException {
    return new Atractie(rs.getInt("Id"), rs.getString("Prefix"), rs.getString("Nume"),
        rs.getString("Judet"), rs.getString("Tip"), rs.getString("Poza"),
        rs.getString("Descriere"), rs.getDouble("Rating"), rs.getDouble("Pret"),
        rs.getDouble("RatioX"), rs.getDouble("RatioY"), rs.getInt("Favorit"));
}
```

...
// Codul continuă în fișierele proiectului predate electronic.

HelloApplication.java

```
package com.example.ghiduristic;

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Scene;
import javafx.stage.Stage;

public class HelloApplication extends Application {
    @Override
    public void start(Stage stage) throws Exception {
```

```
FXMLLoader fxmLoader = new FXMLLoader>HelloApplication.class.getResource("/com/example/ghiduristic/hello-view.fxml"));
stage.setTitle("Ghid Turistic România");
stage.setScene(new Scene(fxmLoader.load(), 1200, 800));
stage.setMaximized(true);
stage.show();
}

public static void main(String[] args) {
    launch();
}
}
```

Judet.java

```
package com.example.ghiduristic;

public class Judet {
    private int id, numarAtractii;
    private String nume, cale;
    private double x, y, w, h;

    public Judet(int id, String nume, String cale, double x, double y, double w, double h) {
        this.id = id; this.nume = nume; this.cale = cale;
        this.x = x; this.y = y; this.w = w; this.h = h;
    }

    public int getId() { return id; }
    public String getNume() { return nume; }
    public String getCale() { return cale; }
    public double getX() { return x; }
    public double getY() { return y; }
    public double getW() { return w; }
    public double getH() { return h; }

    public int getNumarAtractii() { return numarAtractii; }
    public void setNumarAtractii(int numarAtractii) { this.numarAtractii = numarAtractii; }
}
```

MainController

```
package com.example.ghiduristic;

import javafx.fxml.FXML;
import javafx.scene.control.*;
import javafx.scene.image.ImageView;
import javafx.scene.layout.*;
import javafx.scene.Group;
import javafx.scene.shape.Rectangle;
import java.util.*;
import javafx.scene.image.Image;
import javafx.scene.text.*;
import javafx.scene.paint.Color;
import javafx.geometry.Pos;
import javafx.animation.*;
import javafx.util.Duration;
import java.util.prefs.Preferences;

public class MainController {
```

```
    @FXML private ImageView iconitaEarth, pozaRandom, pozaAtractie;
```

```
@FXML private Button btnBack, btnClearSearch, btnVeziFavorite, btnFavorit;
@FXML private Button btnFundal1, btnFundal2, btnFundal3, btnFundal4, btnFundal5;
@FXML private TextField searchBar;
@FXML private ListView<Atractie> listaSearch;
@FXML private VBox panouStart, boxRandom, panouStanga, panouDreapta, containerAtractii;
@FXML private Label lblNumeJudet, numeRandom, titluAtractie, infoAtractie, descriereAtractie;
@FXML private Group mapGroup;
@FXML private StackPane containerHarta;
@FXML private ToggleButton btnIstoric, btnNatura, btnCultura, btnReligios, btnAgrement;

private Map<Atractie, HBox> mapCartonase = new HashMap<>();
private List<Atractie> toateAtractiile = new ArrayList<>();
private Atractie atractieRandomCurenta;
private MapHandler mapHandler;
private DatabaseManager dbManager;
private Image inimaGoala, inimaPlina;
private Preferences preferinteAplicatie;

@FXML
public void initialize() {
    Rectangle clip = new Rectangle();
    clip.widthProperty().bind(containerHarta.widthProperty());
    clip.heightProperty().bind(containerHarta.heightProperty());
    containerHarta.setClip(clip);

    dbManager = new DatabaseManager();
    mapHandler = new MapHandler(mapGroup, dbManager, this);

    inimaGoala = incarcaImagine("icons/empty_heart.png");
    inimaPlina = incarcaImagine("icons/full_heart.png");
    iconitaEarth.setImage(incarcaImagine("icons/earth.png"));

    if (panouDreapta != null) panouDreapta.setVisible(false);

    preferinteAplicatie = Preferences.userNodeForPackage(MainController.class);
    containerHarta.setStyle("-fx-background-color: " + preferinteAplicatie.get("culoareFundal", "#e8f4f8") + ";");

    toateAtractiile = dbManager.getToateAtractiile();
    initPanouStart();
    mapHandler.asambleazaHarta(new ArrayList<>());
}

private Image incarcaImagine(String cale) {
    var resursa = getClass().getResource("/com/example/ghidturistic/" + cale);
    return resursa != null ? new Image(resursa.toExternalForm()) : null;
}

private void schimbaFundalSiSalveaza(String codHex) {
    containerHarta.setStyle("-fx-background-color: " + codHex + ";");
    preferinteAplicatie.put("culoareFundal", codHex);
}

private void initPanouStart() {
    panouStart.setVisible(true); mapGroup.setTranslateX(180);
}

...
// Codul continuă în fișierele proiectului predate electronic.
```

MapHandler.java

```
package com.example.ghidturistic;
```

```
import javafx.scene.image.*;  
import java.util.*;  
import javafx.scene.Group;  
import javafx.animation.*;  
import javafx.util.Duration;  
import javafx.scene.shape.*;  
import javafx.scene.text.*;  
import javafx.scene.paint.Color;  
import javafx.scene.layout.*;  
import javafx.geometry.Pos;  
import javafx.scene.effect.DropShadow;
```

```
public class MapHandler {  
    private Group mapGroup;  
    private Map<String, Image> cacheImagini = new HashMap<>();  
    private Group grupPiniAtractii = new Group();  
    private ImageView judetCurentView;  
    private List<javafx.scene.Node> elementeDeAscuns = new ArrayList<>();  
    private Map<Atractie, ImageView> mapPini = new HashMap<>();  
    private Map<Atractie, Circle> mapPuncteSelectie = new HashMap<>();  
    private Map<String, Text> mapTextNumar = new HashMap<>();  
  
    private HBox panouTooltip = new HBox(4);  
    private ImageView pozaTooltip = new ImageView();  
    private Text numeTooltip = new Text();  
    private DatabaseManager dbManager;  
    private MainController controller;  
    private Atractie atractieSelectata;  
  
    public MapHandler(Group mapGroup, DatabaseManager dbManager, MainController controller) {  
        this.mapGroup = mapGroup; this.dbManager = dbManager; this.controller = controller;  
        panouTooltip.setStyle("-fx-background-color: white; -fx-border-color: #bdc3c7; -fx-border-width: 1; -fx-background-radius: 3; -fx-padding: 3;  
        panouTooltip.setAlignment(Pos.CENTER_LEFT);  
        pozaTooltip.setFitWidth(35); pozaTooltip.setFitHeight(28);  
        numeTooltip.setFont(Font.font("Arial", FontWeight.NORMAL, 9)); numeTooltip.setWrappingWidth(80);  
        panouTooltip.getChildren().addAll(pozaTooltip, numeTooltip);  
        panouTooltip.setVisible(false); panouTooltip.setMouseTransparent(true);  
    }  
  
    private Image incarcaImagine(String cale) {  
        var resursa = getClass().getResource("/com/example/ghidturistic/" + cale);  
        return resursa != null ? new Image(resursa.toExternalForm()) : null;  
    }  
  
    public void assembleazaHarta(List<String> filtre) {  
        mapGroup.getChildren().clear(); mapTextNumar.clear();  
        List<VBox> listaPinuri = new ArrayList<>();  
  
        for (Judet judet : dbManager.incarcaJudete(filtre)) {  
            String url = "imagini/" + judet.getCale();  
            if (!cacheImagini.containsKey(url)) cacheImagini.put(url, incarcaImagine(url));  
  
            ImageView img = new ImageView(cacheImagini.get(url));  
            img.setLayoutX(judet.getX()); img.setLayoutY(judet.getY());  
            img.setFitWidth(judet.getWidth()); img.setFitHeight(judet.getHeight());  
            img.setUserData(judet.getNum()); img.setPickOnBounds(false);
```

```
VBox containerPin = new VBox(2); containerPin.setAlignment(Pos.CENTER); containerPin.setMouseTransparent(true);
Shape pinShape = Shape.union(new Circle(14), new Polygon(-6.0, 10.0, 6.0, 10.0, 0.0, 22.0));
pinShape.setFill(Color.web("#e74c3c")); pinShape.setStroke(Color.WHITE); pinShape.setStrokeWidth(1.5);

Circle cercInt = new Circle(9); cercInt.setFill(Color.WHITE); cercInt.setTranslateY(-5);
Text tNumar = new Text(String.valueOf(judet.getNumarAtractii())); tNumar.setFont(Font.font("Arial", FontWeight.BOLD, 11));
tNumar.setFill(
    mapTextNumar.put(judet.getNume(), tNumar);

Text tNume = new Text(judet.getNume()); tNume.setFont(Font.font("Arial", FontWeight.BOLD, 12)); tNume.setFill(Color.web("#2c3e50"));
DropShadow ds = new DropShadow(); ds.setColor(Color.WHITE); ds.setRadius(3); ds.setSpread(0.85); tNume.setEffect(ds);

containerPin.getChildren().addAll(new StackPane(pinShape, cercInt, tNumar, tNume);
containerPin.setLayoutX(judet.getX() + judet.getWidth() / 2 - 14); containerPin.setLayoutY(judet.getY() + judet.getHeight() / 2 - 38);

img.setOnMouseEntered(e -> { img.setEffect(new javafx.scene.effect.ColorAdjust(0, 0, -0.2, 0)); img.setCursor(javafx.scene.Cursor.HAND); }
img.setOnMouseExited(e -> { img.setEffect(null); img.setCursor(javafx.scene.Cursor.DEFAULT); });

...
// Codul continuă în fișierele proiectului predate electronic.
```

hello-view.fxml

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>
<?import javafx.scene.image.ImageView?>
<?import javafx.scene.image.Image?>
<?import javafx.scene.Group?>

<BorderPane xmlns="http://javafx.com/javafx" xmlns:fx="http://javafx.com/fxml"
    fx:controller="com.example.ghidturistic.MainController"
    prefWidth="1200" prefHeight="800">

    <top>
        <StackPane style="-fx-padding: 10 20; -fx-background-color: #ffffff; -fx-effect: dropshadow(three-pass-box, rgba(0,0,0,0.1), 5, 0, 0, 2);">

            <HBox alignment="CENTER_LEFT" spacing="15" pickOnBounds="false">
                <ImageView fx:id="iconitaEarth" fitWidth="60" fitHeight="60" preserveRatio="true"/>
                <VBox alignment="CENTER_LEFT" spacing="2" pickOnBounds="false">
                    <Label text="Ghid Turistic România" style="-fx-font-size: 24px; -fx-font-weight: bold; -fx-text-fill: #2c3e50;"/>
                    <Label text="Alege un județ pentru a descoperi obiectivele turistice" style="-fx-font-size: 14px; -fx-text-fill: #7f8c8d;"/>
                </VBox>
            </HBox>

            <HBox alignment="CENTER" spacing="15" pickOnBounds="false" translateX="180">
                <Label text="Filtrează atracții:" style="-fx-font-weight: bold; -fx-text-fill: #34495e;"/>

                <ToggleButton fx:id="btnIstoric" onAction="#aplicaFiltre" style="-fx-background-color: white; -fx-background-radius: 20; -fx-border-co
                    <graphic>
                        <HBox alignment="CENTER_LEFT" spacing="8">
                            <ImageView fitWidth="18" fitHeight="18" preserveRatio="true">
                                <image><Image url="@icons/istoric2.png"/></image>
                            </ImageView>
                            <Label text="Istoric" style="-fx-text-fill: #2c3e50; -fx-font-weight: bold;"/>
                        </HBox>
                    </graphic>
                </ToggleButton>
```



```
<ToggleButton fx:id="btnNatura" onAction="#aplicaFiltre" style="-fx-background-color: white; -fx-background-radius: 20; -fx-border-col
<graphic>
  <HBox alignment="CENTER_LEFT" spacing="8">
    <ImageView fitWidth="18" fitHeight="18" preserveRatio="true">
      <image><Image url="@icons/natura2.png"/></image>
    </ImageView>
    <Label text="Natură" style="-fx-text-fill: #2c3e50; -fx-font-weight: bold;"/>
  </HBox>
</graphic>
</ToggleButton>
```

```
<ToggleButton fx:id="btnCultura" onAction="#aplicaFiltre" style="-fx-background-color: white; -fx-background-radius: 20; -fx-border-co
<graphic>
  <HBox alignment="CENTER_LEFT" spacing="8">
    <ImageView fitWidth="18" fitHeight="18" preserveRatio="true">
      <image><Image url="@icons/cultura2.png"/></image>
    </ImageView>
    <Label text="Cultură" style="-fx-text-fill: #2c3e50; -fx-font-weight: bold;"/>
  </HBox>
</graphic>
</ToggleButton>
```

```
<ToggleButton fx:id="btnReligios" onAction="#aplicaFiltre" style="-fx-background-color: white; -fx-background-radius: 20; -fx-border-c
<graphic>
  <HBox alignment="CENTER_LEFT" spacing="8">
    <ImageView fitWidth="18" fitHeight="18" preserveRatio="true">
      <image><Image url="@icons/religios2.png"/></image>
    </ImageView>
    <Label text="Religios" style="-fx-text-fill: #2c3e50; -fx-font-weight: bold;"/>
  </HBox>
</graphic>
</ToggleButton>
```

```
<ToggleButton fx:id="btnAgreement" onAction="#aplicaFiltre" style="-fx-background-color: white; -fx-background-radius: 20; -fx-border-c
<graphic>
  <HBox alignment="CENTER_LEFT" spacing="8">
    <ImageView fitWidth="18" fitHeight="18" preserveRatio="true">
      <image><Image url="@icons/agreement2.png"/></image>
```

...

// Codul continuă în fișierele proiectului predate electronic.